SOLVED ASSIGNMENT

Jenkins CI/CD Pipeline Exercise – Java Maven Project using GitHub & Pipeline as Code

Aim: Create a Jenkins CI/CD pipeline that checks out a Java Maven project from GitHub, builds and tests it, packages it, and performs a deployment step.

This solution follows the uploaded 5-page Jenkins CI/CD Pipeline Guide, including its exact stage names, Jenkins tool names, configuration values, and Declarative Pipeline structure.

1. PIPELINE ARCHITECTURE & FLOW

```
GitHub Repo → Checkout → Build & Test → Deployment
Push/Commit       Jenkins       mvn clean package     Target Server
```

The pipeline starts from a GitHub repository. Jenkins checks out the source code, builds/tests the Maven application, packages it, and then copies the JAR to the deployment destination.

2. STEP 1 – SET UP GITHUB REPOSITORY

Create or fork a simple Java Maven project on GitHub. The repository should contain the following standard layout:

```
my-java-app/
■■■ pom.xml
■■■ src/
    ■■■ main/java/com/example/App.java
    ■■■ test/java/com/example/AppTest.java
```

Copy the repository URL. Example:

```
https://github.com/your-username/my-java-app.git
```

The guide uses **pom.xml** for the Maven configuration, an application source file under **src/main/java**, and a test file under **src/test/java**.

3. STEP 2 – CONFIGURE GLOBAL TOOLS IN JENKINS

Go to **Jenkins Dashboard** → **Manage Jenkins** → **Tools**.

Tool	Name	Installation
JDK	JDK-17	Install automatically
Maven	Maven-3.9	Install automatically

Select automatic installation for both tools and click **Save**.

4. STEP 3 – ADD JENKINSFILE TO GITHUB

Create a file named **Jenkinsfile** in the root of the repository and commit the following Declarative Pipeline:

```
pipeline {
    agent any

    tools {
        maven 'Maven-3.9'
        jdk 'JDK-17'
    }

    stages {
        stage('1. Checkout Code') {
            steps {
                echo 'Fetching source code from GitHub...'
                checkout scm
            }
```

```
        }

        stage('2. Build') {
            steps {
                echo 'Compiling the Java application...'
                sh 'mvn clean compile'
            }
        }

        stage('3. Unit Test') {
            steps {
                echo 'Running unit tests...'
                sh 'mvn test'
            }
            post {
                always {
                    junit '**/target/surefire-reports/*.xml'
                }
            }
        }

        stage('4. Package') {
            steps {
                echo 'Packaging application into JAR/WAR...'
                sh 'mvn package -DskipTests'
            }
        }

        stage('5. Deploy') {
            steps {
                echo 'Deploying application artifact...'
                sh '''
                    echo "Deploying target file to destination server..."
                    cp target/*.jar /tmp/deployed-app.jar || true
                    echo "Deployment completed successfully."
                '''
            }
        }
    }

    post {
        success { echo 'Pipeline completed successfully!' }
        failure { echo 'Pipeline failed. Check the logs for details.' }
    }
}
```

5. EXPLANATION OF JENKINSFILE

Section	Purpose
pipeline	Defines the Jenkins Declarative Pipeline.
agent any	Allows the pipeline to run on any available Jenkins agent.
tools	Provides the predefined JDK-17 and Maven-3.9 installations.
Checkout Code	Fetches the source code from the configured Git repository.
Build	Runs mvn clean compile to compile the Java application.
Unit Test	Runs mvn test and publishes JUnit XML reports.
Package	Runs mvn package -DskipTests to create the application package.
Deploy	Copies target/*.jar to /tmp/deployed-app.jar and prints deployment messages.
post success/failure	Prints a message depending on the final pipeline status.

6. STEP 4 – CREATE AND CONFIGURE JENKINS JOB

1. Open the Jenkins Dashboard and click **New Item**.
2. Enter the item name: **java-mvn-cicd-pipeline**.
3. Select **Pipeline**.
4. In the Pipeline section, select **Pipeline script from SCM**.
5. Select **Git** as SCM.
6. Enter the repository URL:

```
https://github.com/your-username/my-java-app.git
```

7. Set Branch Specifier to:

```
*/main
```

8. Set Script Path to:

```
Jenkinsfile
```

7. STEP 5 – RUN AND VALIDATE THE PIPELINE

1. Click **Build Now** from the left navigation panel.

2. Monitor the pipeline using **Stage View**.

3. Open **Console Output** to inspect complete build and test logs.

The guide illustrates the expected Stage View sequence as:

```
Checkout → Build → Unit Test → Package → Deploy
```

> The guide's example visual shows approximately: Checkout (2s) → Build (12s) → Unit Test (8s) → Package (10s) → Deploy (3s). These are illustrative values from the document, not guaranteed execution times.

8. EXPECTED PIPELINE RESULT

If all stages complete successfully, Jenkins displays the success message:

```
Pipeline completed successfully!
```

If the pipeline fails, Jenkins displays:

```
Pipeline failed. Check the logs for details.
```

The Unit Test stage also publishes JUnit XML reports from:

```
**/target/surefire-reports/*.xml
```

The deployment stage copies the generated JAR to:

```
/tmp/deployed-app.jar
```

9. KEY JENKINS PIPELINE DIRECTIVES

Directive	Description / Purpose
tools { ... }	Binds the predefined JDK and Maven versions into the execution PATH.
checkout scm	Fetches source code automatically from the configured Git repository.
post { junit ... }	Parses JUnit XML report files and publishes test information in Jenkins.
post { success / failure }	Executes actions or messages based on the overall pipeline status.

10. COMPLETE PROCEDURE – SHORT FORM

```
1. Create Java Maven project on GitHub.
2. Ensure pom.xml, App.java and AppTest.java are present.
3. In Jenkins: Manage Jenkins → Tools.
4. Add JDK named JDK-17 with automatic installation.
5. Add Maven named Maven-3.9 with automatic installation.
6. Create Jenkinsfile in the GitHub repository.
7. Add the Declarative Pipeline shown in this assignment.
8. Create Jenkins Pipeline job: java-mvn-cicd-pipeline.
9. Select Pipeline script from SCM → Git.
10. Enter GitHub repository URL.
11. Set branch to */main.
12. Set script path to Jenkinsfile.
13. Click Build Now.
14. Check Stage View and Console Output.
15. Verify Checkout → Build → Unit Test → Package → Deploy.
```

11. RESULT

A Jenkins Declarative CI/CD pipeline was configured for a Java Maven project. The pipeline checks out source code from GitHub, compiles the application, executes unit tests, publishes JUnit reports, packages the application, and performs the deployment step.

12. VIVA / ORAL QUESTIONS AND ANSWERS

Q1. What is Jenkins?
Jenkins is a CI/CD automation server used to automate software build, test and deployment activities.

Q2. What is CI/CD?
CI means Continuous Integration and CD means Continuous Delivery/Deployment. It automates integration, testing, packaging and delivery of software.

Q3. What is a Jenkinsfile?
A Jenkinsfile is a file stored in the repository that defines the Jenkins pipeline as code.

Q4. What is SCM?
SCM means Source Code Management. In this assignment, Git/GitHub is used as the source repository.

Q5. What does checkout scm do?
It fetches the source code from the configured Git repository.

Q6. What does mvn clean compile do?
It removes previous build output and compiles the Java application.

Q7. What does mvn test do?
It executes the project's unit tests.

Q8. Why is junit used in the post block?
It reads JUnit XML test reports and publishes test results in Jenkins.

Q9. What does mvn package -DskipTests do?
It packages the application while skipping test execution at that package stage.

Q10. What is the purpose of the Deploy stage?
It copies the generated JAR to the specified deployment destination.

13. CONCLUSION

The complete GitHub-to-Jenkins CI/CD workflow was established using Pipeline as Code. The project progresses through Checkout, Build, Unit Test, Package and Deploy stages, providing an automated workflow for the Java Maven application.

Note: The uploaded guide provides configuration examples and an illustrative Stage View. Actual build logs, test results, GitHub URL, and execution times must be taken from the student's own Jenkins environment.

— END OF SOLVED ASSIGNMENT —